Chair of IT Security
Brandenburg University of Technology

Winter Semester 2025/2026

Asya Mitseva, M.Sc.
Prof. Dr.-Ing. Andriy Panchenko

Part I: Dataset Analysis and Deep Learning Anomaly Detector Deadline: 12.11.2025

Study Project: Adversarial Machine Learning
Use of Reverse Engineering for Anomaly Detection in Industrial Control Systems

Introduction

Industrial Control Systems (ICS) monitor and control critical infrastructures such as chemical plants, power systems, gas pipelines, and water systems. Typically, ICS consist of two macro areas, known as Information Technology (IT) and Operational Technology (OT). While IT is composed of standard computers interconnected via traditional communication protocols, OT includes special hardware and software for monitoring and controlling a given industrial facility. Overall, OT consists of *field*, *control*, and *supervisory* layers. The field layer includes different sensors and actuators, while the control layer is composed of a number of programmable logic controllers (PLCs). The sensors measure the current state of the underlying physical process, which is then reported to the PLCs. The PLCs implement a control logic used to evaluate the obtained sensor readings according to predefined or dynamically adjustable target values. Based on this evaluation, the PLCs may initiate specific commands to one or more actuators in the field layer aiming to adjust the current state of the physical process. In the supervisory layer, a Supervisory Control and Data Acquisition (SCADA) system performs a high-level visualization and control over the control and field layers and provides an overview of the current process state to the operator. While OT was mainly a standalone system in the past, nowadays it incorporates both decades-old devices and communication infrastructure, and new generation embedded devices with computing capabilities and Ethernet-based communication protocols, and is more often connected to the Internet. While the digitization of ICS makes the monitoring and the control of the physical process easier, it creates new attack surfaces against ICS.

A major line of research focused on the design and the development of robust intrusion detection techniques for the identification of cyber attacks in ICS. A particular challenge to be addressed is the usage of non-standardized protocols that are specific to manufactures or even to a particular type of device. Consequently, protocol specifications needed to process network data are not available and, thus, an in-depth analysis of the content of exchanged network traffic is difficult to accomplish.

Problem Statement

The goal of this study project is to implement and evaluate different approaches that are able to extract expressive patterns, i.e., *features*, from network traffic collected in the OT area of a given ICS. We assume that the ICS operator utilizes proprietary binary protocols, i.e., the approaches to be developed cannot directly parse the content of transmitted network packets. However, the defender can rely on reverse engineering methods to derive only specific chunks from the observed network packets. We rely on different machine learning (ML) methods to train a classifier with these chunks in order to create a classification model. Finally, the defender uses the generated model to identify a possible abnormal operation in ICS.

In the first part of the study project, we get familiar with three popular ICS datasets, namely

Electra (S7comm)[1] and QUT_S7Comm[2], analyze the properties of these datasets, and implement our first anomaly detection method.

Task 1: Statistical Analysis of ICS Network Traffic

The goal of this task is to get familiar with two popular ICS datasets, called Electra (S7comm) and QUT_S7Comm. Both datasets are public ICS datasets that were collected from different experimental testbeds. For this task, you should only consider the network traffic in the form of PCAP files to answer the questions below. Please note that you are not allowed to use any parsed physical readings collected from the Historian of the testbed or any other data format such as the data that has already been extracted from the network traffic by the authors of the datasets. In this task, you should write a piece of code that collects the following general-purpose statistics for this dataset:

a) How many network packets does each dataset contain in total? How many of them are under attack and how many are normal packets? How many and which application-layer network protocols are present in each dataset? What is the fraction of packets exchanged though each of these protocols with respect to the total number of packets in each dataset; what is the fraction of packets under attack exchanged though each of these protocols? Which transport-layer protocols are present in each dataset? If the identified number of transport-layer protocols is more than one, what is the fraction of packets exchanged though each of these protocols with respect to the total number of packets in each dataset; what is the fraction of packets under attack exchanged though each of these protocols? If the identified number of transport-layer protocols is more than one, what is the fraction of packets for each application-layer protocol exchanged though each of the transport-layer protocols in each dataset; what is the fraction of packets under attack for each application-layer protocol exchanged though each of the transport-layer protocols?

b) What is the average total packet length (in bytes) for each protocol type when you consider only the network packets that contain application data for each of both: packet with and without attacks? What is the standard deviation and the median of packet length (in bytes) for each protocol type when you consider only the network packets that contain application data for each of both: packet with and without attacks? What is the average inter-arrival time between two consecutive packets that are exchanged between the same pair of hosts for each of both: packet with and without attacks? What is the standard deviation and the median of inter-arrival time between two consecutive packets that are exchanged between the same pair of hosts for each of both: packet with and without attacks?

c) How many pairs of hosts can be found in each dataset? For how many of these pairs are there packets with attacks? What is the average inter-arrival time between two consecutive packets that are exchanged between the same pair of hosts and are transmitted over the same application-layer protocol type without attacks? What is the standard deviation and the median of inter-arrival time between two consecutive packets that are exchanged between the same pair of hosts and are transmitted over the same application-layer protocol type without attacks? What is the average inter-arrival time between two consecutive packets that are exchanged between the same pair of hosts and are transmitted over the same application-layer protocol type with attacks? What is the standard deviation and the median of inter-arrival time between two consecutive packets that are exchanged between the same pair of hosts and are transmitted over the same application-layer protocol type with attacks?

d) Plot two cumulative distribution functions (CDFs) for the header length (in bytes) of network packets per application-layer protocol type and for the application payload length

[1] `http://perception.inf.um.es/ICS-datasets/csv/electra_s7comm.zip`
[2] `https://github.com/qut-infosec/2017QUT_S7comm`

(in bytes) of network packets per application-layer protocol type in each dataset with and without attacks. Please note that you are not allowed to use any preexisting library for any part of the computation of the CDFs. Not only the source code, but also the generated plots should be submitted in Moodle.

should we use only the LabeledDatase because it is the one with pcap files.

Task 2: Similarity between Different Datasets

The goal of this task is to analyze the similarity of the provided network traffic between the different files, in which it was collected. To this end, you need to write a piece of code that satisfies the following requirements:

should we read all of the pcap files? or some of them?

this should be done

a) For each of your datasets, you need to generate traffic flows from the corresponding network traffic. Each flow should correspond to a time window of $\{2, 4, 6\}$ minutes and should only contain traffic exchanged between two communicating parties over the same highest-level protocol. From the corresponding sequence of packets belonging to a given flow, you should extract only packet size, direction, and inter-arrival packet time that will be used for further analysis. All flows should have the same length. Please make sure that you do not lose information for longer flows. You should differentiate between flows containing packets under attack and those without attacks.

b) Using the flows generated for each of the time intervals, your piece of code should compute the Chebyshev distance between every two generated flows per time interval. For the computation of Chebyshev distance, you are not allowed to use any existing libraries, i.e., the programming logic should be implemented by you. You should compute Chebyshev distance between either two flows without attacks or two flows with attack (but not between flow with attack and flow without attack).

c) You should write a piece of code that creates six different histograms for each of the flow time windows and for the flows with and without attacks plotting the different distributions of the computed Chebyshev distances. For the computation of the histograms, you are not allowed to use any existing function from a library, i.e., the programming logic of creating the histogram should be implemented by you. Not only the source code, but also the generated plot should be submitted in Moodle.

d) You should write a piece of code that creates six different plots for each of the flow time windows and for the flows with and without attacks plotting the *t-distributed stochastic neighbor embedding (t-SNE)* for the different files, i.e., a single color in a plot should represent the flows from a single file. Please implement and conduct hyperparamerter tuning for t-SNE before you generate the final plots. Not only the source code, but also the generated plot should be submitted in Moodle.

e) For each of your datasets, you should compute the Chebyshev distance between every two raw packets (represented as a sequence of bytes). For the computation of Chebyshev distance, you are not allowed to use any existing libraries, i.e., the programming logic should be implemented by you. You should compute Chebyshev distance between either two packets without attacks or two packets with attack (but not between packet with attack and packet without attack).

f) You should write a piece of code that creates two different histograms for the set of packets with and without attacks plotting the different distributions of the computed Chebyshev distances for single packets. For the computation of the histograms, you are not allowed to use any existing function from a library, i.e., the programming logic of creating the histogram should be implemented by you. Not only the source code, but also the generated plot should be submitted in Moodle.

Task 3: Unsupervised Detection of Network Attacks

The goal of this task is to write a piece of code that implements a deep learning (DL) based detection method classifying each input from a given dataset as either being benign, i.e. normal, or malicious, i.e., containing an attack. In particular, you should implement a Stacked Denoising autoencoder (SDA) that takes m x n values as an input and returns n features as an output. The autoencoder should be able to work with network packets as an input data, i.e., it should take as an input m x n network packets, where m indicates the number of packets and n the set of bytes of each packet. The autoencoder should be trained with data from the normal operation of a given ICS in order to optimize the Mean Squared Error. In particular, your piece of code should satisfy the following requirements:

a) Your autoencoder model should take as an input a predefined set of bytes n_p per network packet such that $n_p = M$ values and M should be a parameter that is read from the command line.

b) Your SDA model should support *at least* two different types of layers and *at least* three different activation functions (i.e., it should create different SDA models for the different layers and the different activation functions). Make sure that for each SDA model you provide an implementation for the search of optimal hyperparameters.

c) For each of the created autoencoder models, your piece of code should be able to print out the features obtained after training for each of the different VAE models in a file.

d) You should further extend your autoencoder model to be able to work as a classifier as well. However, you are not allowed to use the classifier function for the generation of features. The classifier function should satisfy the following requirement: Given a target vector of values $\vec{r}_s$ and the corresponding encoded vector $\vec{o}_s$, we define $\vec{e} = \vec{r}_s - \vec{o}_s = [d_1, ..., d_n]$ as the reconstruction error n-dimensional vector and $avg(\vec{e}) = \frac{1}{n} \cdot \sum_{n=1}^{n} d_i^2$ as the corresponding average reconstruction error. Finally, the predicted value $y(\vec{r}_s)$ for an input $\vec{r}_s$ should be *under attack* if and only if $avg(\vec{e}) > \gamma$, where γ is a threshold predefined by the user.

Preparation for Q&A Session

Prepare yourself for a Q&A session of 40 minutes. During the Q&A session, you need to give an overview of libraries, scripts or already existing tools that you have used. Furthermore, you need to make a demo of your piece of code, explain its operation in detail, and show all plots and data that you generated. Last but not least, you should be ready to answer spontaneous questions asked by the reviewer.